

Apache Iceberg Delta Lake Module Architecture Design Document

Delta Lake to Iceberg Table Migration Module
Architecture / Design Patterns / Call Chain Analysis

Project: **Apache Iceberg 1.9.2**

Module: **iceberg-delta-lake**

Type: **Architecture Design Document**

Date: **2026-03-23**

Table of Contents

1 Module Overview

- 1.1 Purpose and Positioning
- 1.2 Source File Overview

2 Architecture Design

- 2.1 Overall Architecture Diagram
- 2.2 Layer Design Description
- 2.3 Migration Execution Flowchart
- 2.4 Sequence Diagram

3 Design Pattern Analysis

- 3.1 Builder Pattern (Fluent API)
- 3.2 Visitor Pattern (Type Conversion)
- 3.3 Strategy Pattern (Commit Classification)
- 3.4 Provider / Factory Pattern
- 3.5 Transaction Pattern
- 3.6 Immutable Value Pattern

4 Core Class Design Details

- 4.1 SnapshotDeltaLakeTable Interface
- 4.2 BaseSnapshotDeltaLakeTableAction
- 4.3 TypeVisitor & TypeToType
- 4.4 MigrationActionsProvider

5 Key Scenario Call Chains

- 5.1 Complete Snapshot Migration
- 5.2 Initial Version Commit (Fault Tolerance)
- 5.3 VersionLog Incremental Commit
- 5.4 Type System Conversion
- 5.5 Data File Construction
- 5.6 Spark Integration

6 Summary and Design Merits

1. Module Overview

1.1 Purpose and Positioning

The iceberg-delta-lake module is a dedicated migration tool within the Apache Iceberg project ecosystem. Its core function is to snapshot an existing Delta Lake table and convert it to an Iceberg table, enabling organizations to seamlessly transition their data lake infrastructure from Delta Lake to Iceberg format.

Key design goals of this module:

- Zero-Copy Migration: No data files are duplicated. Only Iceberg metadata is created pointing to original Parquet files.
- Version-Faithful Replay: Every Delta Lake version log is replayed one-by-one into corresponding Iceberg snapshots.
- Tag-based Traceability: Each Iceberg snapshot is tagged with Delta version number and timestamp for lineage tracing.
- Transactional Safety: All operations wrapped in a single Iceberg Transaction ensuring atomicity.
- Engine Agnostic Core: Core logic is engine-independent; Spark integration is a thin convenience layer.

1.2 Source File Overview

The module contains 5 main source files with approximately 600 lines of core logic:

File Name	Type	Responsibility
SnapshotDeltaLakeTable	Interface	Public API: fluent builder + Result type
BaseSnapshotDeltaLake...	Class	Core: read Delta log, convert schema, replay
DeltaLakeDataTypeVisitor	Abstract	Generic Visitor for Delta type hierarchy
DeltaLakeTypeToType	Class	Visitor impl: Delta -> Iceberg types
...ActionsProvider	Interface	Provider/Factory with Singleton default
...SparkIntegration	Utility	Spark: resolve catalog from SparkSession

Table 1: Source Files of the delta-lake Module

2. Architecture Design

2.1 Overall Architecture Diagram

The module follows a clean four-tier layered architecture:

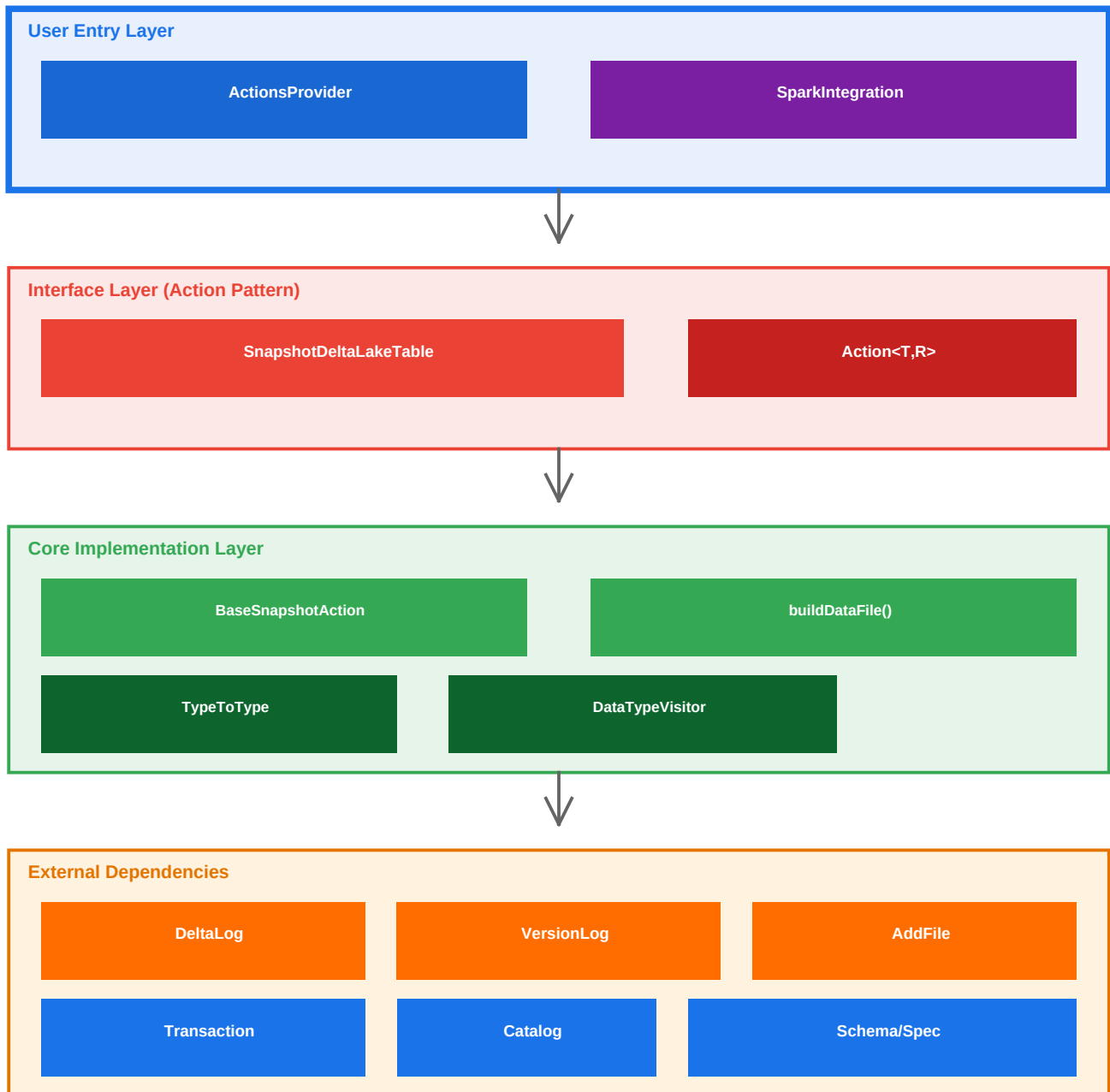


Figure 1: Module Architecture

2.2 Layer Design Description

- User Entry Layer: Two entry points - MigrationActionsProvider for programmatic use, SparkIntegration for Spark SQL context. Both create a SnapshotDeltaLakeTable action.
- Interface Layer: SnapshotDeltaLakeTable extends Action<ThisT, R>. Defines fluent builder methods and the

execute() contract.

- Core Implementation: BaseSnapshotDeltaLakeTableAction orchestrates schema conversion (via Visitor pattern), version log replay, and transactional commit.
- External Dependencies: Delta Lake Standalone (DeltaLog, VersionLog, AddFile, RemoveFile) for reading Delta metadata; Iceberg Core API (Transaction, Catalog, Schema) for creating the target table.

2.3 Migration Execution Flowchart

The execute() method follows an 11-step sequential workflow:



Figure 2: Migration Execution Flowchart

2.4 Sequence Diagram

Interaction between core participants during execute():

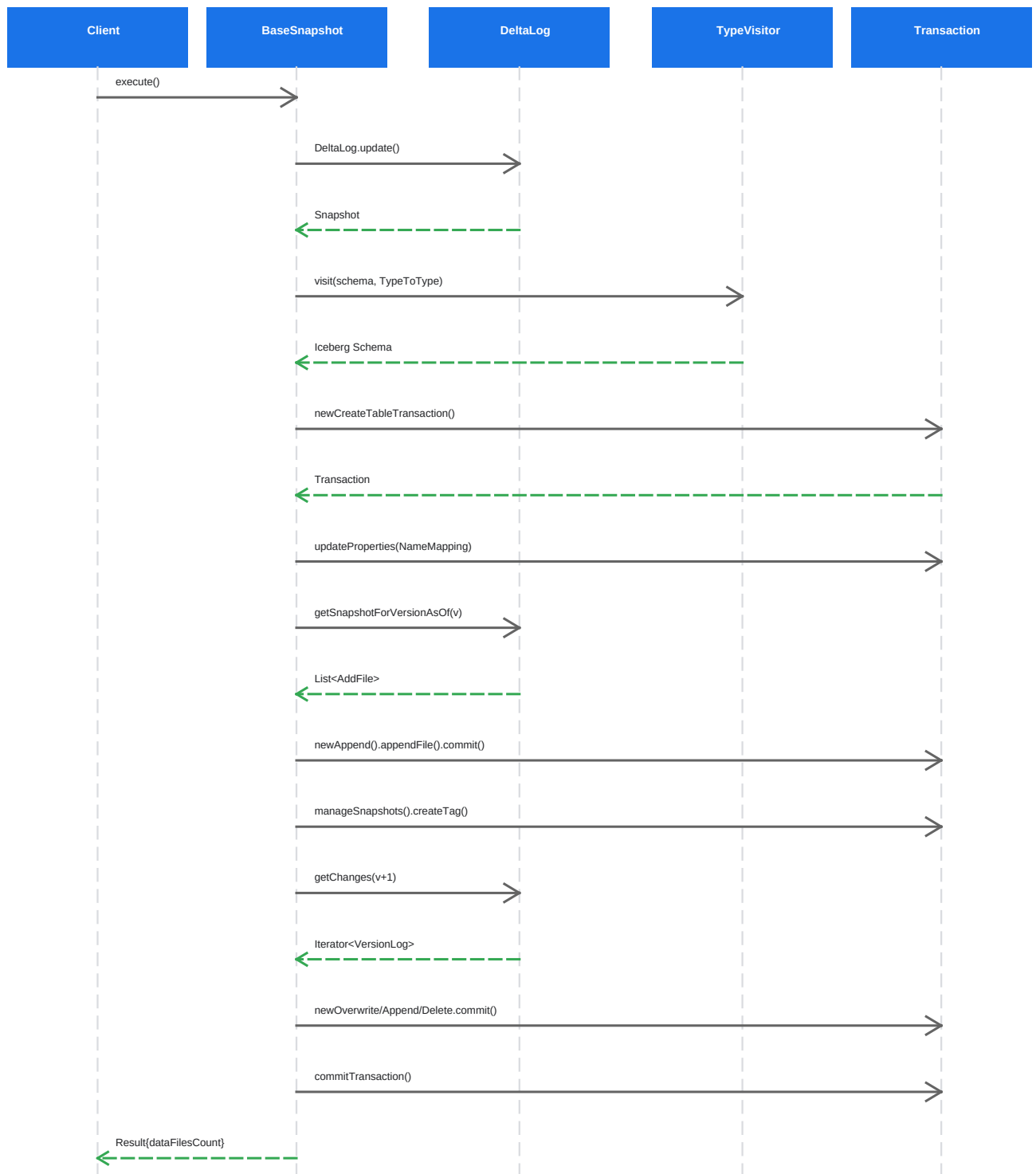


Figure 3: Sequence Diagram - execute()

3. Design Pattern Analysis

The delta-lake module employs six well-known design patterns, each solving a specific architectural concern.

3.1 Builder Pattern (Fluent API)

Pattern Intent: Separate construction from representation, enabling step-by-step configuration with method chaining.

Application: SnapshotDeltaLakeTable defines fluent builder methods where each returns "this", allowing natural chaining:

```
snapshotDeltaLakeTable(location)
  .as(TableIdentifier.of("db", "table"))
  .icebergCatalog(catalog)
  .deltaLakeConfiguration(hadoopConf)
  .tableProperty("key", "value")
  .tableLocation(newLocation)
  .execute();
```

Benefit: Self-documenting API. The execute() method validates all required parameters via Preconditions.checkArgument. Optional parameters have sensible defaults.

3.2 Visitor Pattern (Type Conversion)

Pattern Intent: Perform operations on elements of an object structure without modifying those classes.

Application: DeltaLakeDataTypeVisitor<T> is an abstract visitor that traverses Delta Lake's type tree. DeltaLakeTypeToType is the concrete visitor that converts each Delta type node into an Iceberg Type.

Type Mapping Table:

Delta Lake Type	Iceberg Type
BooleanType	BooleanType
IntegerType / ShortType / ByteType	IntegerType
LongType	LongType
FloatType	FloatType
DoubleType	DoubleType
StringType	StringType
DateType	DateType
TimestampType	TimestampType.withZone()
DecimalType(precision, scale)	DecimalType.of(p, s)
BinaryType	BinaryType

Benefit: Clean separation of traversal and conversion logic. Adding new Delta types only requires extending atomic(). Recursive visit() mechanism handles nested types automatically.

3.3 Strategy Pattern (Commit Classification)

Pattern Intent: Define interchangeable algorithms encapsulated in each commit strategy.

Application: `commitDeltaVersionLogToIcebergTransaction` classifies each `VersionLog` into one of four strategies:

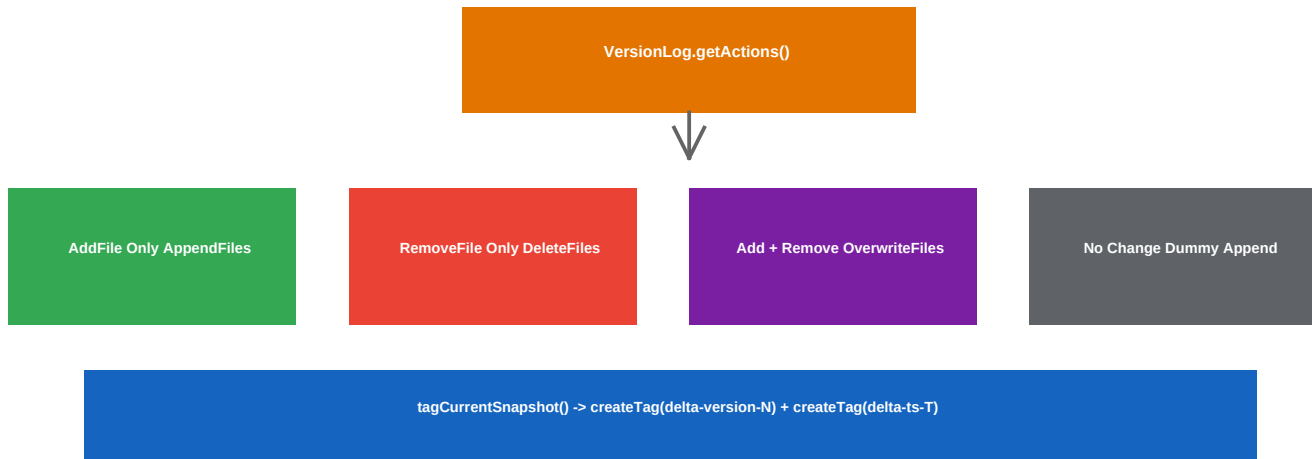


Figure 4: Commit Classification

3.4 Provider / Factory Pattern

`DeltaLakeToIcebergMigrationActionsProvider` serves as both provider interface and factory. It defines a default method `snapshotDeltaLakeTable()` that instantiates `BaseSnapshotDeltaLakeTableAction`. The inner class `DefaultDeltaLakeToIcebergMigrationActions` implements the Singleton pattern. Query engines can override this provider to supply custom implementations.

3.5 Transaction Pattern

The entire migration is wrapped in a single Iceberg Transaction created via `icebergCatalog.newCreateTableTransaction()`. All operations (append, delete, overwrite, property update, tag) are committed within this transaction. Only when `commitTransaction()` succeeds does the new Iceberg table become visible.

3.6 Immutable Value Pattern

The `SnapshotDeltaLakeTable.Result` interface is annotated with `@Value.Immutable` (Immutables library), generating `ImmutableSnapshotDeltaLakeTable.Result` at compile time. This pattern is consistent with Iceberg's project-wide convention.

4. Core Class Design Details

4.1 SnapshotDeltaLakeTable Interface

Public API entry point extending `Action<SnapshotDeltaLakeTable, Result>`.

Method	Return	Purpose
<code>tableProperties(Map)</code>	this	Batch set Iceberg table properties
<code>tableProperty(name, val)</code>	this	Set a single table property
<code>tableLocation(location)</code>	this	Override new table location (default: Delta location)
<code>as(TableIdentifier)</code>	this	Set new Iceberg table identifier [Required]
<code>icebergCatalog(Catalog)</code>	this	Set target Iceberg catalog [Required]
<code>deltaLakeConfiguration()</code>	this	Set Hadoop conf for Delta access [Required]
<code>execute()</code>	Result	Execute migration, return result summary

4.2 BaseSnapshotDeltaLakeTableAction

The most critical class in the module (~460 lines, package-private). Key internal fields:

- `deltaLog (DeltaLog)`: Delta Lake transaction log, initialized via `DeltaLog.forTable()`
- `icebergCatalog (Catalog)`: Target Iceberg catalog for creating the new table
- `deltaLakeFileIO (HadoopFileIO)`: File I/O for reading data files and collecting metrics
- `deltaStartVersion (long)`: Earliest available version (may not be 0 after VACUUM)
- `additionalPropertiesBuilder (ImmutableMap.Builder)`: Accumulates user-specified properties

Key Design Decision - Fault Tolerant Initial Version: The method `commitInitialDeltaSnapshotToIcebergTransaction` handles VACUUM-cleaned tables by iterating from `deltaStartVersion` upward until finding a constructable snapshot. Catches `NotFoundException` and `DeltaStandaloneException` to skip inaccessible versions.

4.3 DeltaLakeDataTypeVisitor and DeltaLakeTypeToType

`DeltaLakeDataTypeVisitor<T>`: Abstract class with static `visit()` dispatch. Recursively traverses Delta type tree (struct -> field -> atomic/array/map). Five abstract methods: `struct()`, `field()`, `array()`, `map()`, `atomic()`.

`DeltaLakeTypeToType` extends visitor (`T=Iceberg Type`). Manages `nextId` counter for field IDs. Root fields use ordinal-based IDs; nested fields use auto-incrementing IDs. Correctly handles nullable vs. required semantics and preserves comments from Delta metadata.

4.4 DeltaLakeToIcebergMigrationActionsProvider

Combines Provider + Factory + Singleton patterns. Defines default method `snapshotDeltaLakeTable()` returning new `BaseSnapshotDeltaLakeTableAction()`. Inner class `DefaultDeltaLakeToIcebergMigrationActions` is a private singleton. Static method `defaultActions()` returns this singleton. Engines can override the provider with custom implementations.

5. Key Scenario Call Chains

5.1 Complete Snapshot Migration (Primary Scenario)

```
Client -> MigrationActionsProvider.defaultActions()
-> new BaseSnapshotDeltaLakeTableAction(location)
-> .as(identifier) .icebergCatalog(catalog) .deltaLakeConfiguration(conf)
-> DeltaLog.forTable(conf, location) // init deltaLog
-> new HadoopFileIO(conf) // init file IO
-> deltaLog.getVersionAtOrAfterTimestamp(0) // find start version
-> .execute()
-> Preconditions.checkArgument(...) // validate all params
-> deltaLog.update() // get latest snapshot
-> convertDeltaLakeSchema(deltaSchema)
    -> DeltaLakeDataTypeVisitor.visit(schema, new DeltaLakeTypeToType(schema))
-> getPartitionSpecFromDeltaSnapshot(schema, snapshot)
-> icebergCatalog.newCreateTableTransaction(...)
-> transaction.table().updateProperties().set(NameMapping).commit()
-> commitInitialDeltaSnapshotToIcebergTransaction(version, tx)
-> [loop] commitDeltaVersionLogToIcebergTransaction(vlog, tx)
-> icebergTransaction.commitTransaction()
-> return ImmutableResult{snapshotDataFilesCount}
```

5.2 Initial Version Commit (Fault Tolerance)

```
commitInitialDeltaSnapshotToIcebergTransaction(latestVersion, tx)
-> constructableStartVersion = deltaStartVersion
-> [while constructableStartVersion <= latestVersion]
-> deltaLog.getSnapshotForVersionAsOf(version).getAllFiles()
-> [for each AddFile]
    -> buildDataFileFromAction(addFile, table)
        -> getFullFilePath(path, deltaLog.getPath())
        -> deltaLakeFileIO.newInputFile(fullPath)
        -> file.exists() ? // validate file presence
        -> ParquetUtil.fileMetrics(file, metricsConfig, nameMapping)
        -> DataFiles.builder(spec).withPath().withMetrics().build()
-> transaction.newAppend().appendFile(each).commit()
-> tagCurrentSnapshot(version, tx)
    -> manageSnapshots().createTag("delta-version-N")
    -> manageSnapshots().createTag("delta-ts-T")
-> return version // success
-> [catch NotFoundException | DeltaStandaloneException]
-> constructableStartVersion++ // try next version
```

5.3 VersionLog Incremental Commit

```
commitDeltaVersionLogToIcebergTransaction(versionLog, tx)
-> versionLog.getActions().stream()
  -> filter(action instanceof AddFile || RemoveFile)
-> [for each action] buildDataFileFromAction(action, table)
  -> classify into filesToAdd / filesToRemove
-> [decision branch]
  -> !filesToAdd.empty AND !filesToRemove.empty
    -> tx.newOverwrite().addFile(each).deleteFile(each).commit()
  -> !filesToAdd.empty
    -> tx.newAppend().appendFile(each).commit()
  -> !filesToRemove.empty
    -> tx.newDelete().deleteFile(each).commit()
  -> else
    -> tx.newAppend().commit() // dummy append
-> tagCurrentSnapshot(versionLog.getVersion(), tx)
```

5.4 Type System Conversion

```
DeltaLakeDataTypeVisitor.visit(deltaStructType, visitor)
-> [type is StructType]
-> for each StructField:
  -> visit(field.getDataType(), visitor) // RECURSIVE
  -> [atomic] visitor.atomic(type)
    -> DeltaLakeTypeToType.atomic(): map to Iceberg type
  -> [array] visit(elementType) -> visitor.array(type, elem)
    -> Types.ListType.ofOptional/ofRequired(nextId, elementType)
  -> [map] visit(keyType), visit(valueType) -> visitor.map()
    -> Types.MapType.ofOptional/ofRequired(nextId, nextId, k, v)
  -> [struct] RECURSIVE into nested struct
-> visitor.field(field, typeResult)
-> visitor.struct(structType, fieldResults)
-> assign field IDs (ordinal for root, auto-increment for nested)
-> preserve nullable/required semantics
-> extract comment from field metadata
-> return Types.StructType.of(newFields)
```

5.5 Data File Construction

```
buildDataFileFromAction(action, table)
-> [instanceof AddFile]
  -> extract path, size, partitionValues from AddFile
-> [instanceof RemoveFile]
  -> extract path, size (Optional), partitionValues from RemoveFile
-> getFullFilePath(path, deltaLog.getPath())
  -> URI.create(path) -> decode -> absolute? return : prepend tableRoot
-> Preconditions.checkArgument(partitionValues != null)
-> determineFileFormatFromPath(path) // must be .parquet
-> deltaLakeFileIO.newInputFile(fullPath)
-> file.exists() ? : throw NotFoundException
-> fileSize = nullableFileSize ?? file.getLength()
-> MetricsConfig.forTable(table)
-> NameMappingParser.fromJson(nameMapping)
-> ParquetUtil.fileMetrics(file, metricsConfig, nameMapping)
-> DataFiles.builder(spec)
  .withPath(fullPath).withFormat(PARQUET)
  .withFileSizeInBytes(fileSize).withMetrics(metrics)
  .withPartitionValues(partitionValueList).build()
```

5.6 Spark Integration Scenario

```
DeltaLakeToIcebergMigrationSparkIntegration.snapshotDeltaLakeTable(spark, id, loc)
-> Preconditions.checkArgument(spark, id, loc != null)
-> spark.sessionState().catalogManager().currentCatalog()
-> Spark3Util.catalogAndIdentifier(ctx, spark, id, defaultCatalog)
-> DeltaLakeToIcebergMigrationActionsProvider.defaultActions()
  -> .snapshotDeltaLakeTable(deltaTableLocation)
    -> new BaseSnapshotDeltaLakeTableAction(location)
  -> .as(TableIdentifier.parse(catalogAndIdent.identifier()))
  -> .deltaLakeConfiguration(spark.sessionState().newHadoopConf())
  -> .icebergCatalog(Spark3Util.loadIcebergCatalog(spark, catalogName))
-> return SnapshotDeltaLakeTable // ready for .execute()
```

6. Summary and Design Merits

The iceberg-delta-lake module is a masterclass in focused, well-designed migration tooling. Key architectural merits:

- Minimal Footprint, Maximum Impact: With only 5 main source files and ~600 lines of core logic, the module achieves complete, version-faithful Delta-to-Iceberg migration.
- Zero-Copy Data Philosophy: No data files are copied or transformed. Only Iceberg metadata is created pointing to existing Parquet files, making migration instantaneous regardless of data volume.
- Robust Fault Tolerance: The initial version search mechanism gracefully handles VACUUM-cleaned and log-cleaned Delta tables, automatically finding the first constructable version.
- Full History Preservation: Every Delta version is replayed as a distinct Iceberg snapshot, with version and timestamp tags enabling time-travel queries on the migrated table.
- Transactional Atomicity: The entire migration is wrapped in a single Iceberg transaction. If any step fails, no partial table is left behind.
- Clean Separation of Concerns: Type conversion (Visitor), action creation (Provider/Factory), commit strategy (Strategy), and configuration (Builder) each have dedicated, single-responsibility classes.
- Extensibility: The Provider pattern allows engine overrides. The Visitor pattern enables easy type additions. The commit classification can be extended with new Action type handlers.
- Consistency with Iceberg Conventions: Package-private implementations, Immutables for value objects, Preconditions for validation, Action<ThisT, R> interface pattern, and NameMapping for schema evolution.

Overall, this module demonstrates that well-chosen design patterns and a clear architectural vision can produce a highly functional migration tool with minimal code complexity, while maintaining the high quality standards expected of the Apache Iceberg project.